

Curso de Go

Aula 19 - Desenvolvendo um Microserviço Simples em Go

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 **códigos_**
BRAZUCA ^[3]





Nessa Aula nos Vamos Ver

- **Nesta aula vamos desenvolver a estrutura básica de um microserviço utilizando Go.**
- **Principais conceitos:**
 - **Organização do projeto**
 - **MVC simplificado**
 - **Separação de responsabilidades**
 - **Criação de endpoints HTTP**
 - **Manipulação de dados**
 - **Respostas em JSON**
- **Parte Prática**



O que é um Microserviço?

- **Um microserviço é uma aplicação pequena responsável por executar uma função específica dentro de um sistema.**
- **Exemplos:**
 - **Serviço de usuários**
 - **Serviço de produtos**
 - **Serviço de pagamentos**
 - **Serviço de pedidos**
 - **Serviço de autenticação**
- **Cada microserviço pode possuir sua própria API, regras de negócio e banco de dados.**



Características de um Microserviço

- **Um microserviço normalmente possui:**
- **Responsabilidade bem definida**
- **API própria**
- **Baixo acoplamento**
- **Independência de outros serviços**
- **Comunicação através de HTTP ou mensageria**
- **Possibilidade de implantação independente**
- **Em nosso exemplo criaremos um pequeno serviço responsável por produtos.**



Estrutura do Projeto

- Uma possível estrutura simplificada:

```
microservice/  
├── main.go  
├── controllers/  
│   └── product_controller.go  
├── models/  
│   └── product.go  
└── services/  
    └── product_service.go
```

Essa organização permite separar as diferentes responsabilidades da aplicação.



MVC Simplificado

- O MVC (Model-View-Controller) é um padrão de organização de software que separa as responsabilidades da aplicação em diferentes componentes.
- Em um microserviço desenvolvido em Go, normalmente utilizamos uma versão simplificada desse padrão: o Model representa as estruturas de dados, como produtos, usuários ou pedidos; o Controller recebe e responde às requisições HTTP, como GET, POST, PUT e DELETE; e a camada de Service, frequentemente utilizada no lugar da View tradicional, concentra as regras de negócio.
- Essa separação torna o código mais organizado, facilita a manutenção, os testes e a evolução do microserviço.



MVC Simplificado

- **Em APIs REST, normalmente não temos uma View tradicional com páginas HTML.**
- **Podemos utilizar uma adaptação:**
 - **Model**
 - **Controller**
 - **Service**
- **O JSON retornado pela API funciona como representação dos dados.**



Model

- O Model representa os dados utilizados pela aplicação.
- Exemplo:

```
package models

type Product struct {
    ID      int      `json:"id"`
    Name    string   `json:"name"`
    Price   float64  `json:"price"`
}
```

- Essa estrutura representa um produto dentro do microserviço.
- As tags json determinam os nomes utilizados na resposta JSON.



Camada de Service

- A camada Service contém regras de negócio e operações relacionadas aos dados.Exemplo:

```
package services
```

```
import "microservice/models"
```

```
var products = []models.Product{  
    {ID: 1, Name: "Notebook", Price:  
3500},  
    {ID: 2, Name: "Mouse", Price: 100},  
}
```

```
func GetProducts() []models.Product {  
    return products  
}
```

- O controller não precisa saber como os produtos são armazenados.



Controller

- O Controller recebe as requisições HTTP.
- Ele normalmente é responsável por:
- Receber requisições
- Validar parâmetros
- Chamar os Services
- Definir códigos HTTP
- Retornar JSON
- Exemplo:

```
func GetProducts(w http.ResponseWriter,  
r *http.Request) {  
    products := services.GetProducts()  
  
    w.Header().Set("Content-Type",  
"application/json")  
  
    json.NewEncoder(w).Encode(products)  
}
```



Separação de Camadas

- Podemos visualizar a aplicação da seguinte maneira:

```
Cliente
|
v
Controller
|
v
Service
|
v
Model
```

- Cada camada possui uma responsabilidade específica.
- Isso evita colocar toda a aplicação dentro do arquivo main.go.



Arquivo main.go

- O arquivo main.go será responsável principalmente por iniciar o servidor e registrar as rotas.

```
package main

import (
    "fmt"
    "net/http"

    "microservice/controllers"
)

func main() {

    http.HandleFunc("/products", controllers.GetProducts)

    fmt.Println("Servidor rodando em http://localhost:8080")

    http.ListenAndServe(":8080", nil)
}
```

O main.go funciona como ponto inicial da aplicação.



Criando o Primeiro Endpoint

- Um endpoint representa um endereço disponibilizado pela API.
- Exemplo:

`GET /products`

- Acessando:

`http://localhost:8080/products`

- A API poderia retornar:

```
[  
  {  
    "id": 1,  
    "name": "Notebook",  
    "price": 3500  
  },  
  {  
    "id": 2,  
    "name": "Mouse",  
    "price": 100  
  }  
]
```




Principais Métodos HTTP

- **Uma API REST normalmente utiliza os seguintes métodos:**

GET
POST
PUT
DELETE

- **Exemplos:**

GET /products
GET /products/1
POST /products
PUT /products/1
DELETE /products/1

Cada método representa uma operação diferente sobre os recursos.



Endpoint GET

- O método **GET** é utilizado para consultar informações.
- **Exemplo:**

`GET /products`

- **Retorna todos os produtos.**
- **Outro exemplo:**

`GET /products/1`

- **Retorna apenas o produto com ID igual a 1. O servidor normalmente retorna o código:**

`200 OK`



Endpoint POST

- O método POST é utilizado para criar novos recursos.
- Exemplo de requisição:

```
POST /products
```

- JSON enviado:

```
{  
  "name": "Teclado",  
  "price": 250  
}
```

- No Go podemos receber os dados utilizando:

```
var product models.Product  
  
json.NewDecoder(r.Body).Decode(&product)
```

- Depois o Service pode adicionar o produto à lista ou ao banco de dados.



Códigos HTTP

- Uma API deve informar corretamente o resultado da operação.
- Alguns códigos importantes:

200 OK

201 Created

400 Bad Request

404 Not Found

500 Internal Server Error

- **Exemplo**

```
w.WriteHeader(http.StatusCreated)
```

- **Ou**

```
http.Error(  
    w,  
    "Produto não encontrado",  
    http.StatusNotFound,  
)
```

- O código HTTP permite que o cliente saiba se a operação foi realizada com sucesso.



Arquitetura Final do Microserviço

- Ao final teremos uma estrutura semelhante a:

```
Cliente
|
| HTTP
v
Endpoints
|
v
Controllers
|
v
Services
|
v
Models
```

- Exemplo de Endpoints

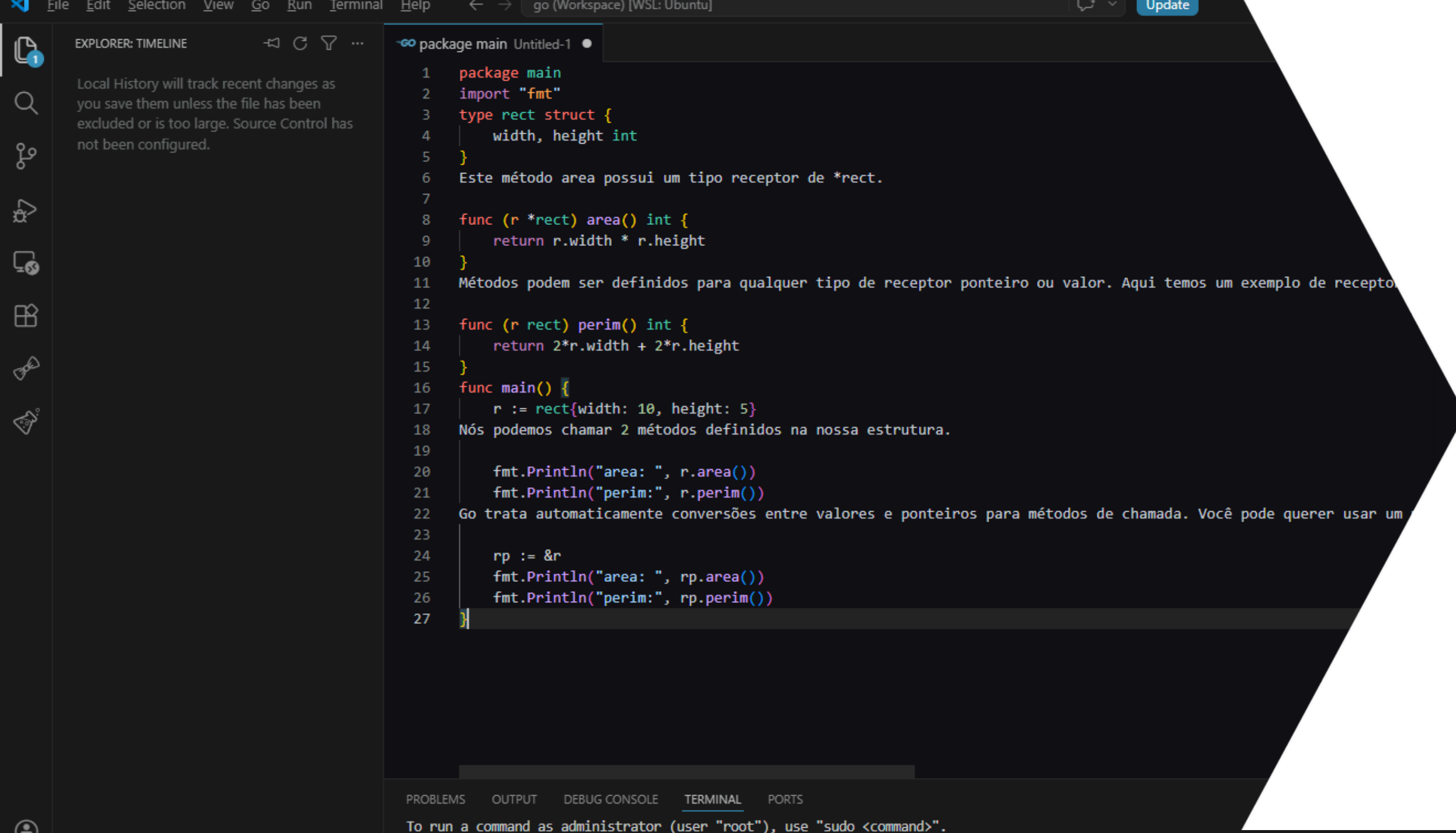
```
GET    /products
GET    /products/{id}
POST   /products
PUT    /products/{id}
DELETE /products/{id}
```

- Com essa estrutura conseguimos desenvolver um microserviço Go simples, organizado e preparado para posteriormente adicionar banco de dados, autenticação, logs, middlewares e testes.



Boas práticas

- **Responsabilidade única:** cada microserviço deve resolver um problema específico e bem definido.
- **Separação de camadas:** organize o código em Models, Controllers, Services e outras camadas quando necessário.
- **APIs bem definidas:** utilize endpoints claros e siga padrões REST para facilitar a comunicação.
- **Baixo acoplamento:** evite dependências excessivas entre diferentes microserviços.
- **Tratamento de erros:** retorne mensagens e códigos HTTP adequados, como 200, 400, 404 e 500.
- **Logs e monitoramento:** registre operações importantes e erros para facilitar diagnóstico e manutenção.
- **Segurança:** implemente autenticação, autorização, validação de dados e comunicação segura.
- **Testes automatizados:** crie testes unitários e de integração para garantir o funcionamento do serviço.



The screenshot shows a Go IDE with a file named 'package main' in 'Untitled-1'. The code defines a struct 'rect' with 'width' and 'height' of type 'int'. It includes two methods: 'area()' which returns the product of width and height, and 'perim()' which returns the perimeter (2*width + 2*height). The 'main' function creates a 'rect' with width 10 and height 5, prints its area and perimeter, and then uses pointers to call these methods. Comments in Portuguese explain the methods and the use of pointers. The IDE interface includes a sidebar with icons for Explorer, Search, Source Control, Run and Debug, Extensions, and Testing. The bottom status bar indicates the user is running as administrator.

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de receptor
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Parte Prática

Desenvolvendo um Microserviço Simples em Go